Actividad Evaluativa Eje3
Polimorfismo en Programación Orientada a Objetos

Alumno : Daniel Alejandro Sarmiento Villamarin
Profesor: Claudia Patricia Ramírez Triana

Modelos de Programación 1 6C-71
Fundación Universitaria del Área Andina
2026

Introducción

La programación orientada a objetos permite un programa mediante clases y objetos . Entre sus conceptos principales se encuentran la herencia y el polimorfismo. En este taller se desarrolla NotificaPro , una aplicación de consola escrita en JavaScript que simula la preparación de notificaciones por correo electrónico , SMS Y WhatsApp.

La clase Notificación contiene los datos comunes y el método enviar() . La clase CorreoElectrónico, SMS Y WhatsApp heredan de ella y redefinen ese método para producir una respuesta particular según el canal seleccionado. De esta manera, una misma llamada al método puede tener comportamientos diferentes de acuerdo con el objeto.

Objetivos

Objetivo General

Desarrollar una aplicación en JavaScript que demuestre el uso del polimorfismo mediante un método común redefinido en varias clases derivadas.

Objetivos Específicos

- Identificar una clase base y sus clases derivadas.
- Representar mediante un diagrama UML la relación de herencia entre las clases.
- Implementar el método enviar() con comportamientos distintos en cada canal de notificación.
- Probar la aplicación desde la terminal y explicar los resultados obtenidos.
- Relacionar la solución desarrollada con los conceptos de herencia y polimorfismo.

Descripción del problema y propuesta de solución

Se requiere representar un sistema sencillo que permita preparar una notificación mediante uno de tres canales: correo electrónico, SMS o WhatsApp. Aunque todos los canales comparten el nombre del destinatario, el dato de contacto y el mensaje, la forma en que se identifica el canal y se describe el resultado cambia.

La solución consiste en crear una clase general llamada Notificacion y tres clases especializadas. Cada clase hija utiliza super() para aprovechar el constructor de la clase base y redefine el método enviar(). La aplicación solicita datos al usuario, crea el objeto correspondiente y ejecuta notificacion.enviar() sin necesitar una llamada distinta para cada clase.

Conceptos de programación orientada a objetos

Clase

Una clase es una plantilla que define los datos y comportamientos de los objetos. En este proyecto, Notificacion es la clase base y las otras tres clases describen canales específicos.

Herencia

La herencia permite que una clase derivada reutilice atributos y métodos de otra clase. En JavaScript se expresa con la palabra extends. La instrucción super() invoca el constructor de la clase padre.

Polimorfismo

El polimorfismo permite utilizar una misma operación con objetos de diferentes clases y obtener comportamientos particulares. En NotificaPro, el método enviar() existe en la clase base y se redefine en CorreoElectronico, SMS y WhatsApp. La instrucción notificacion.enviar() ejecuta la implementación asociada al tipo real del objeto.

Diseño de clases y diagrama UML

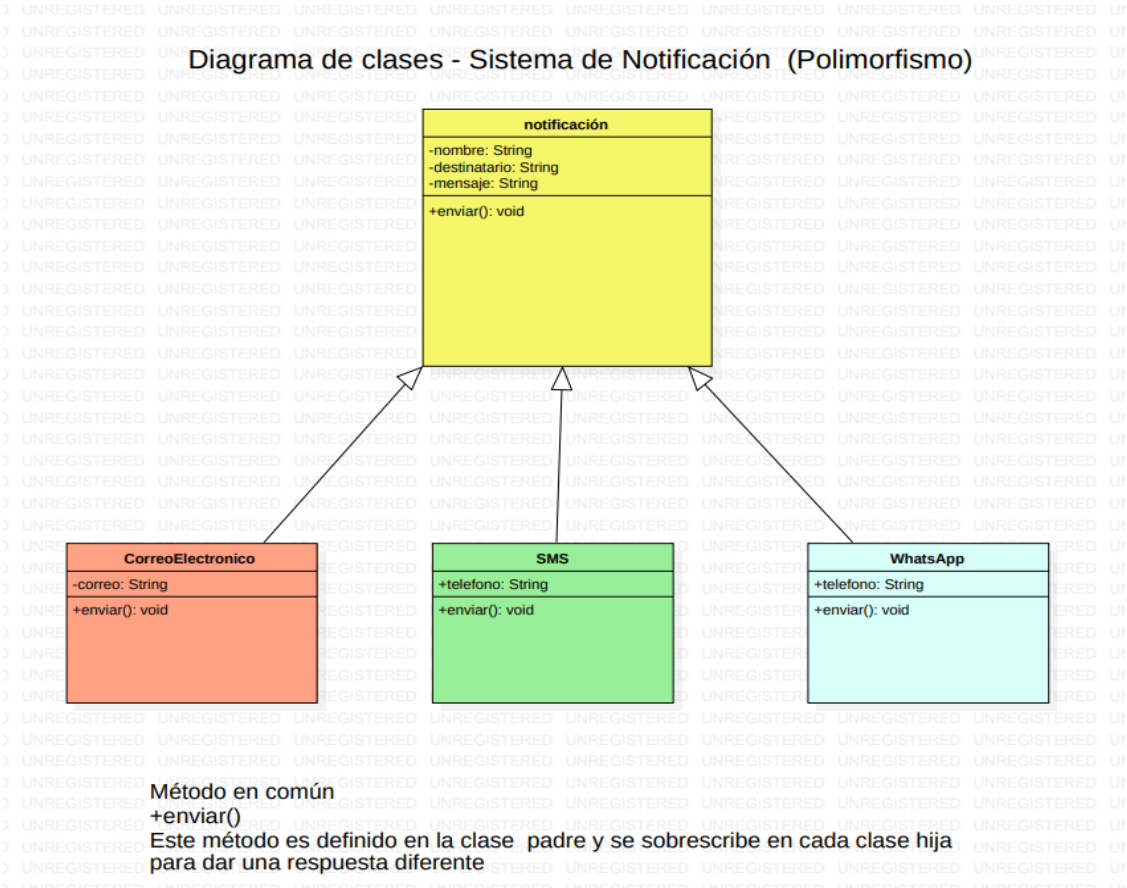
El sistema contiene las siguientes clases:

• Notificacion: clase padre. Contiene nombre, destinatario y mensaje, además del método enviar().

• CorreoElectronico: clase hija que representa una notificación por correo.

• SMS: clase hija que representa una notificación por mensaje de texto.

• WhatsApp: clase hija que representa una notificación por WhatsApp. Las tres clases hijas se conectan con Notificacion mediante una relación de generalización (herencia). Cada una redefine enviar().



se adjunta diagrama e image de igual manera en el repositorio lo encontrara en formato pdf para una mejor visualización

Código fuente de la aplicación

El archivo notificaciones.js contiene el código ejecutable. Se adjunta por separado para que pueda abrirse en Visual Studio Code y ejecutarse con Node.js

```
const readline = require("readline");


const colores = {
    reset: "\x1b[0m",
    azul: "\x1b[34m",
    cian: "\x1b[36m",
    verde: "\x1b[32m",
    amarillo: "\x1b[33m",
    rojo: "\x1b[31m",
    morado: "\x1b[35m",
    negrita: "\x1b[1m"
};

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});


function titulo(texto) {
    console.log("\n" + colores.cian + colores.negrita);
    console.log("========================================");
    console.log("       " + texto);
    console.log("========================================");
    console.log(colores.reset);
}


function preguntar(texto) {
    return new Promise((resolver) => {
        rl.question(colores.amarillo + texto + colores.reset, (respuesta) =>
{
            resolver(respuesta.trim());
        });
    });
}


class Notificacion {
```

```
    constructor(nombre, destinatario, mensaje) {
        this.nombre = nombre;
        this.destinatario = destinatario;
        this.mensaje = mensaje;
    }

    enviar() {
        return "Procesando notificación...";
    }
}


class CorreoElectronico extends Notificacion {
    enviar() {
        return {
            canal: "Correo electrónico",
            icono: "📧",
            destinatario: this.destinatario,
            mensaje: this.mensaje,
            detalle: "Correo enviado correctamente."
        };
    }
}


class SMS extends Notificacion {
    enviar() {
        return {
            canal: "Mensaje SMS",
            icono: "💬",
            destinatario: this.destinatario,
            mensaje: this.mensaje,
            detalle: "Mensaje SMS enviado correctamente."
        };
    }
}


class WhatsApp extends Notificacion {
    enviar() {
        return {
            canal: "WhatsApp",
            icono: "📱",
            destinatario: this.destinatario,
            mensaje: this.mensaje,
            detalle: "Mensaje de WhatsApp enviado correctamente."
```

```
        };
    }
}


async function iniciarAplicacion() {
    console.clear();

    console.log(colores.morado + colores.negrita);
    console.log("╔═══════════════════════════════════════╗");
    console.log("║          NOTIFICAPRO                  ║");
    console.log("║      Sistema de notificaciones        ║");
    console.log("╚═══════════════════════════════════════╝");
    console.log(colores.reset);

    console.log(colores.cian + "\nIngrese los datos del destinatario\n" +
colores.reset);

    const nombre = await preguntar("  Nombre completo: ");
    const correo = await preguntar("  Correo electrónico: ");
    const telefono = await preguntar("  Número de teléfono: ");
    const mensaje = await preguntar("  Escriba su mensaje: ");

    titulo("SELECCIONE EL CANAL");

    console.log(colores.azul + "  [1] 📧 Correo electrónico");
    console.log("  [2] 💬 Mensaje SMS");
    console.log("  [3] 📱 WhatsApp" + colores.reset);

    const opcion = await preguntar("\n  Seleccione una opción (1-3): ");

    let notificacion;

    switch (opcion) {
        case "1":
            notificacion = new CorreoElectronico(
                nombre,
                correo,
                mensaje
            );
            break;

        case "2":
            notificacion = new SMS(
                nombre,
```

```
                telefono,
                mensaje
            );
            break;

        case "3":
            notificacion = new WhatsApp(
                nombre,
                telefono,
                mensaje
            );
            break;

        default:
            console.log(colores.rojo + "\n  Opción no válida." +
colores.reset);
            rl.close();
            return;
    }


    const resultado = notificacion.enviar();

    titulo("RESUMEN DE LA NOTIFICACIÓN");

    console.log(colores.verde + "  ✓ " + resultado.detalle + colores.reset);
    console.log("\n  Nombre:       " + nombre);
    console.log("  Canal:        " + resultado.canal);
    console.log("  Destinatario: " + resultado.destinatario);
    console.log("  Mensaje:      " + resultado.mensaje);

    console.log(colores.cian +
"\n========================================");
    console.log("       PROCESO FINALIZADO");
    console.log("========================================" + colores.reset);

    console.log(colores.amarillo +
        "\nNota: esta aplicación es una simulación." +
        "\nNo se envían mensajes reales." +
        colores.reset);

    rl.close();
}

iniciarAplicacion().catch((error) => {
```

```
    console.error("Ocurrió un error:", error);
    rl.close();
});
```

El archivo completo adjunto incluye la interfaz de consola, la lectura de datos, la selección del canal, validaciones básicas y la presentación del resultado. La aplicación es demostrativa: no se conecta a servidores ni envía mensajes reales.

Descripción del funcionamiento

1. La aplicación muestra el título NotificaPro y explica que las notificaciones son simuladas.

2. Solicita el nombre del destinatario y el texto del mensaje.

3. Permite seleccionar correo electrónico, SMS o WhatsApp.

4. Solicita una dirección de correo o un número telefónico, según el canal.

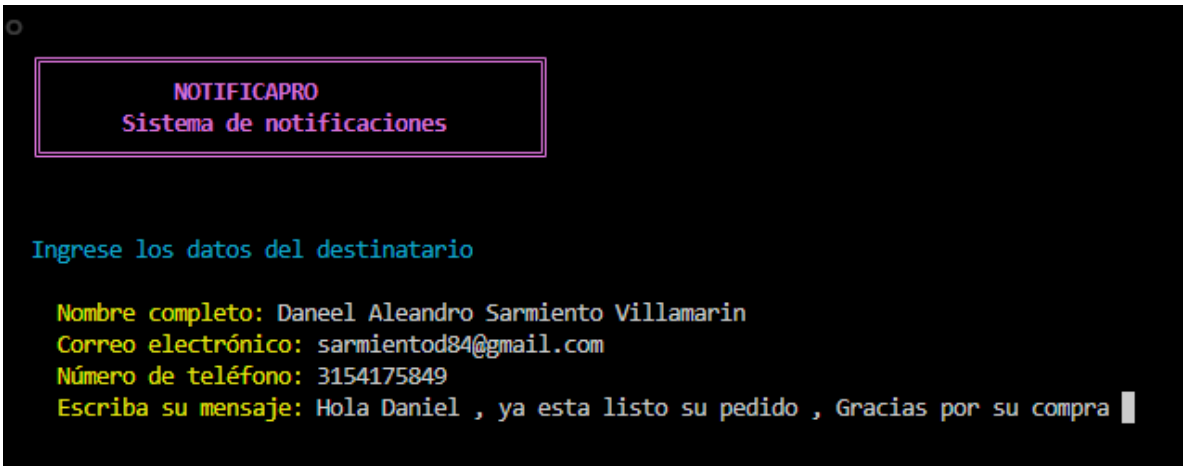
5. Construye un objeto de la clase correspondiente. 6. Invoca el método común enviar() y presenta el resultado específico del canal.

Demostración del polimorfismo

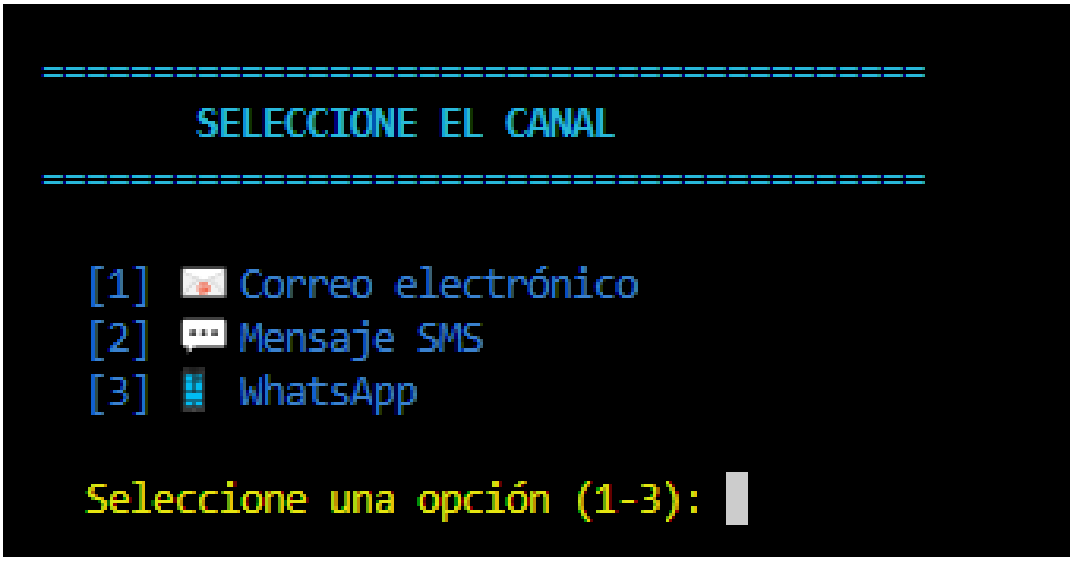
La demostración principal es la instrucción const resultado = notificacion.enviar(). La variable notificacion puede contener un objeto CorreoElectronico, SMS o WhatsApp. No obstante, la llamada al método conserva el mismo nombre. JavaScript ejecuta la versión redefinida en la clase concreta, por lo que el canal y el detalle del resultado cambian. La herencia evita repetir los atributos nombre, destinatario y mensaje en cada clase. La redefinición del método permite que cada canal proporcione una respuesta especializada.

Evidencias

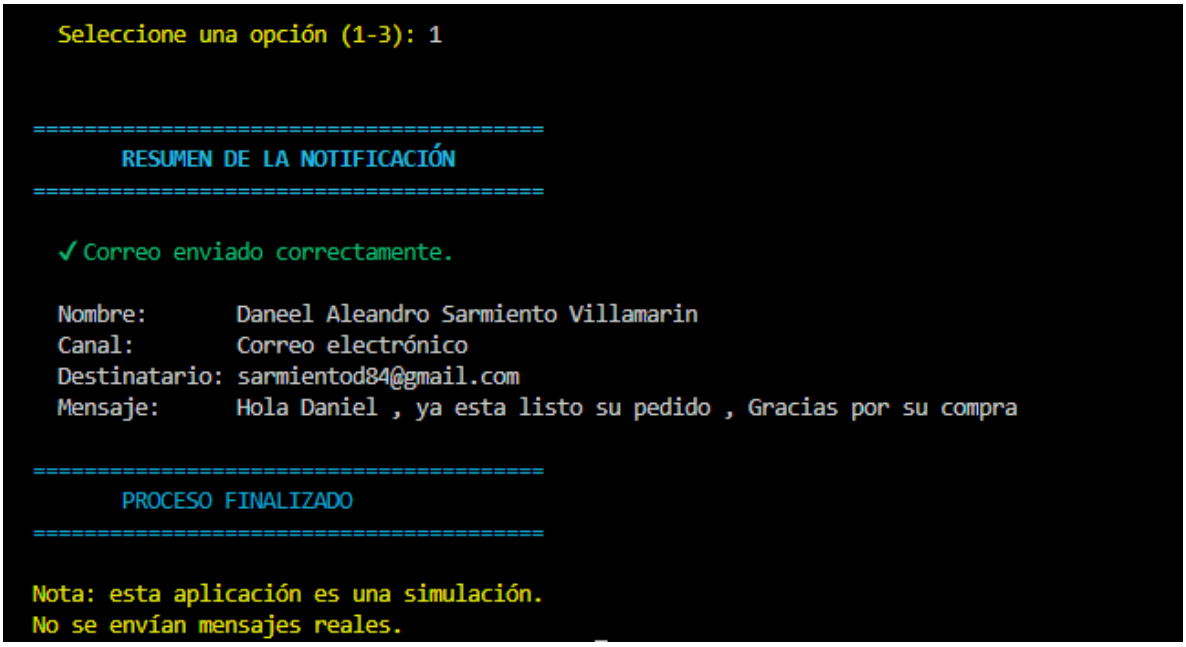
Introducimos los datos del destinatario y el mensaje



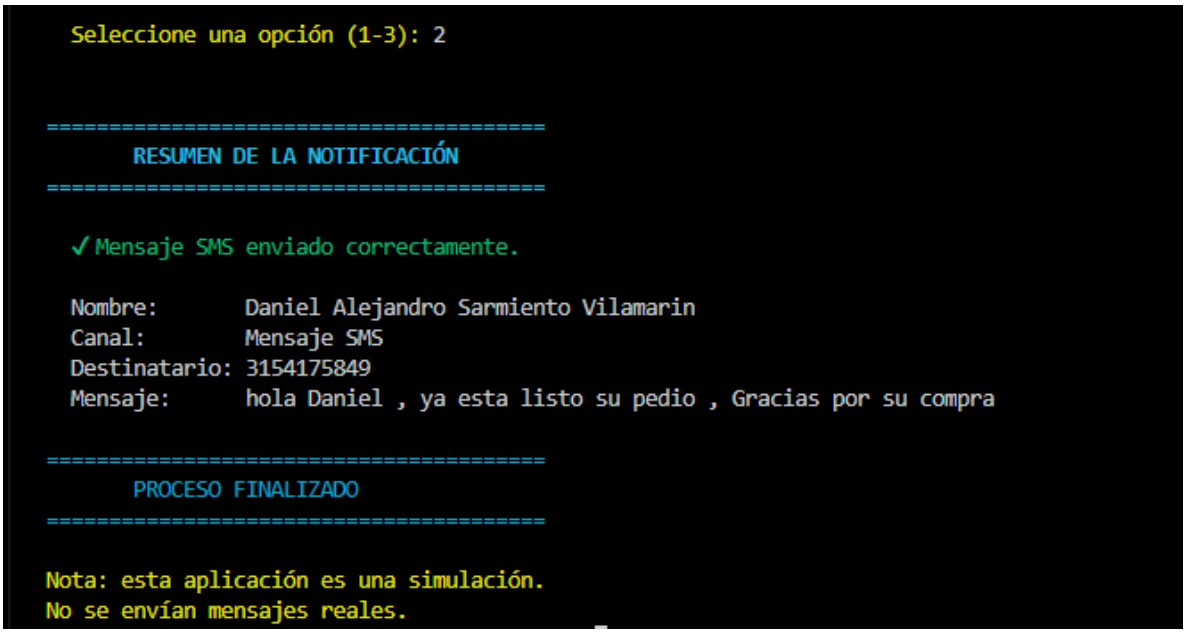
Seleccionamos el canal



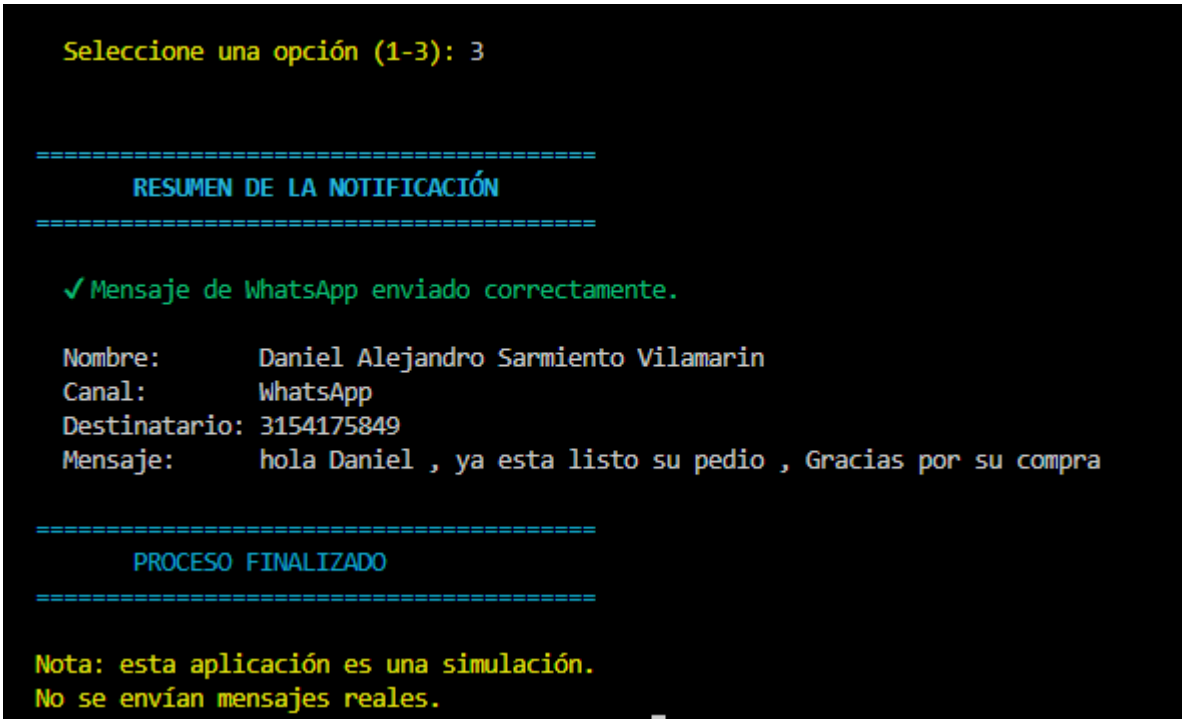
Resumen de la notificación del canal seleccionado



Ejemplo del canal de SMS



Ejemplo de canal WhatsApp



Conclusiones

El desarrollo de NotificaPro permitió aplicar conceptos de programación orientada a objetos en JavaScript. La clase Notificacion centraliza los datos comunes, mientras que las clases derivadas representan canales de comunicación específicos.

Se comprobó el polimorfismo mediante el método enviar(), cuyo nombre se mantiene en las diferentes clases, aunque su resultado cambia según el objeto. Esto facilita ampliar el programa: un nuevo canal podría incorporarse creando otra clase derivada que redefina el mismo método.

Finalmente, la práctica reforzó el uso de clases, constructores, herencia con extends, reutilización con super() y redefinición de métodos. Como mejora futura, la simulación podría integrarse con servicios reales de mensajería, siempre que se cuente con las credenciales, permisos y configuración necesarios.

Link de trabajo en Git

https://github.com/alejofla2/actividadeje3

(En el link encontramos el código y el diagrama en pdf)

Referencias bibliográficas

Curso de JavaScript (Manual Imprescindible)

Autor: David Solini El gran libro de Node.js

Autor: Jordi Colell Eloquent JavaScript

Autor: Marijn Haverbeke

Canales de Yotube :

Holamundo

MoureDev By Moure

Programador X

TodoCode